

Claims Copilot — Architecture & New Domain Guide

Overview

Claims Copilot is a plugin-based AI claims examiner workflow platform. The core files (`core/`, `api.py`, `main.py`, `agents`, `frontend`) contain **zero domain literals**. A domain is a single folder implementing `DomainPlugin`. Adding a new client domain requires creating one folder and adding one import line — nothing else changes.

Directory Structure

```
claims-copilot/
├── core/
│   ├── contracts.py      # DomainPlugin + DomainConfig dataclasses (the contract)
│   ├── registry.py       # DOMAIN_REGISTRY dict + get_active_plugin()
│   └── engine.py         # Generic runners: run_rules, score_claim, run_checklist
├── domains/
│   ├── __init__.py       # Bootstrap: one "import domains.<name>" per domain
│   └── car/
│       └── __init__.py    # Car plugin assembly: PLUGIN = DomainPlugin(...)
├── intelligence/         # Car-domain rules, scoring, checklist, vision
│   ├── rules_engine_car.py # CAR_RULE_SPECS: List[RuleSpec] + shim run_car_rules_engine
│   ├── risk_scoring_car.py # CAR_FEATURE_SPECS: List[FeatureSpec] + shim score_car_claim
│   ├── checklist_car.py    # CHECKLIST: List[Step] (single source of truth)
│   └── document_vision.py  # Vision analysis; reads _get_vision_prompt() lazily from plugin
├── agents/
│   ├── copilot.py         # Session-managed copilot; reads PLUGIN lazily
│   ├── nodes.py          # LangGraph investigation/dossier nodes; reads PLUGIN lazily
│   ├── prompts_car.py     # Car copilot system prompt
│   ├── prompts_investigation_car.py # Car orchestrator / investigation / dossier prompts
│   ├── tools_car.py       # Car copilot tools (set_context + ALL_CAR_TOOLS)
│   ├── tools_investigation_car.py # Car investigation tools
│   └── tools_dossier_car.py # Car dossier tools
├── data/
│   └── generate_car.py     # Synthetic car data generator
├── cases.py               # Neutral Case dataclass (no domain fields)
├── domain_config.py       # CAR_INSURANCE = DomainConfig(...) (car config data)
├── case_queue_car.py      # build_car_case_queue(claims, scores, rules, data) -> List[Case]
├── main.py               # initialize_data() - plugin-driven, stamped per-domain cache
├── api.py                 # FastAPI - all domain reads via get_active_plugin()
├── frontend/src/
│   ├── App.jsx           # Fetches /api/config; sidebar uses domainConfig.display_name
│   ├── components/
│   │   ├── CaseQueue.jsx # TYPE_LABELS/COLORS/tabs from /api/config
│   │   ├── ChatPanel.jsx # WebSocket URL via window.location.host (relative)
│   │   ├── DossierPanel.jsx # Neutral dossier template; relative API URLs
│   │   └── RiskDashboard.jsx # Relative /api/claims/stats fetch
│   └── hooks/
│       ├── useCopilotChat.js # WS URL via window.location.host
│       └── useInvestigation.js # WS URL via window.location.host
└── tests/
    ├── fixtures/golden_baseline.json # Hero-case regression snapshots
    ├── test_golden_baseline.py       # 14 regression tests (scores, tiers, rules, queue order)
    ├── test_rules_engine.py          # 25 unit tests for R-001 to R-006
    ├── test_plugin_conformance.py    # 24 contract assertions over all registered plugins
    └── test_domain_swap.py           # 10 smoke tests: init + queue, no phantom fields
```

The Plugin Contract (`core/contracts.py`)

```
@dataclass(frozen=True)
class DomainPlugin:
    name: str # matches DOMAIN_MODE env var, e.g. "car_insurance"
    config: DomainConfig

    # Data + intelligence
    generate_fn: Callable[[], Dict] # () -> dict with context_keys as top-level keys
    rules_fn: Callable # (claim, context_dict) -> List[RuleResult]
    score_fn: Callable # (claim, context_dict, rules) -> RiskBreakdown
    build_queue_fn: Callable # (claims, scores, rules, data_dict) -> List[Case]
    checklist_module: Any # exposes run_full_checklist, run_checklist_step, CHECKLIST_STEPS

    # Agent prompts + tools
    copilot_prompt: str
    copilot_tools: List
    investigation_prompts: Dict[str, str] # keys: ORCHESTRATOR, INVESTIGATION, DOSSIER
    investigation_tools: List
    dossier_tools: List

    # Vision + UI
    vision_prompt: str
    checklist_descriptions: Dict[int, str] # step_number -> one-liner description

    # Entity wiring
    context_keys: List[str] # keys generate_fn returns, e.g. ["claims", "insureds", ...]
    build_checklist_ctx: Callable # (data_ctx, claim_id) -> dict for checklist module
    dossier_context_fn: Callable # (data_ctx, claim_id) -> dict for dossier builder
    set_tool_context_fns: List[Callable] # called at startup: [tools.set_context, ...]
    entity_types: List[str] # mirrors config.entity_types
```

`DomainConfig` (same file) holds pure data — display name, claim types, prefixes, risk tiers, workflow task types, checklist step names, system references, entity types.

Data Flow

```
DOMAIN_MODE env var
|
v
domains/__init__.py (imports domains.car → registers PLUGIN)
|
v
core/registry.py DOMAIN_REGISTRY["car_insurance"] = PLUGIN
|
v
main.py
    initialize_data()
    PLUGIN.generate_fn() → raw data dict
    PLUGIN.rules_fn() → claim_rules dict
    PLUGIN.score_fn() → claim_risk_scores dict
    PLUGIN.build_queue_fn() → case_queue List[Case]
    pickle to data_cache_{PLUGIN.name}.pkl
    (stamped as (domain_name, ctx) tuple)
|
v
api.py lifespan
    for fn in PLUGIN.set_tool_context_fns: fn(data_ctx)
|
v
/api/config
    returns display_name, claim_types, type_labels, type_colors,
    entity_labels, risk_tiers, checklist_steps
|
v
frontend
    App.jsx fetches /api/config on mount
    CaseQueue uses type_labels/colors/entity_labels from config
    DossierPanel uses neutral template (subject_name, asset_description, etc.)
```

The Case Dataclass (cases.py) — Neutral Fields Only

```
@dataclass
class Case:
    case_id: str
    case_type: CaseType
    claim_type: ClaimType
    subject_id: str          # neutral: was insured_id (car), member_id (health)
    subject_name: str
    priority: CasePriority
    flag_reason: str
    risk_score: float = 0.0
    claim_source: str = "company_site"
    status: CaseStatus = CaseStatus.NEW
    created_at: datetime = ...
    date_filed: datetime = ...
    summary: Optional[str] = None
    key_metrics: Dict = ...
    rules_triggered: List[Dict] = ...
    workflow_tasks: List[Dict] = ...
    document_flags: List[Dict] = ...
    checklist_state: Dict = ...
    asset_description: str = ""    # neutral: vehicle, property, or asset under claim
    counterparty_name: str = ""    # neutral: repair shop, vendor, or opposing party
    claim_amount: float = 0.0
    coverage_start: Optional[str] = None
    coverage_end: Optional[str] = None
    investigation_history: List[Dict] = ...
    dossier: Optional[str] = None
```

Phantom fields that must NEVER appear in a Case: member_id , employer_name , provider_name

Cache Stamp Protocol (main.py)

```
cache_file = f"data_cache_{PLUGIN.name}.pkl"
# Write: pickle.dump((domain_name, ctx), f)
# Read:  cached = pickle.load(f)
#        if isinstance(cached, tuple) and cached[0] == domain_name:
#            ctx = cached[1]    # domain matches — use it
#        else:
#            regenerate()      # stale cache from a different domain
```

/api/config Response Shape

```
{
  "name": "car_insurance",
  "display_name": "Car Insurance Claims Examiner Copilot",
  "claim_types": ["collision", "comprehensive", "theft", "liability", "medical_payments"],
  "type_labels": {"collision": "Collision", "theft": "Theft", ...},
  "type_colors": {"collision": {"bg": "rgba(59,130,246,0.12)", "text": "#60a5fa"}, ...},
  "risk_tiers": {"HIGH": [60, 100], "MEDIUM": [30, 59], "LOW": [0, 29]},
  "checklist_steps": ["Initial Review", "Damage Documentation", ...],
  "entity_labels": {"insured": "Insured", "vehicle": "Vehicle", "repair_shop": "Repair Shop"}
}
```

All frontend rendering is driven by this response. No domain strings are hardcoded in the React components.

Adding a New Domain — Step-by-Step Guide

Step 1 — Create the domain folder

```
domains/  
  my_domain/  
    __init__.py    ← required: assemblies and registers PLUGIN
```

Step 2 — Implement `DomainConfig`

```
from core.contracts import DomainConfig  
  
MY_DOMAIN_CONFIG = DomainConfig(  
    name="my_domain",  
    display_name="My Domain Claims Examiner",  
    claim_types=["type_a", "type_b"],  
    claim_id_prefixes={"type_a": "TA", "type_b": "TB"},  
    claim_sources=["online_portal", "call_center"],  
    claim_source_weights={"online_portal": 0.7, "call_center": 0.3},  
    risk_tiers={"HIGH": (60, 100), "MEDIUM": (30, 59), "LOW": (0, 29)},  
    workflow_task_types=["INITIAL_REVIEW", "DOCUMENT_REQUEST"],  
    checklist_steps=["Initial Review", "Document Check", "Final Determination"],  
    system_references={},  
    rules_prefix="R",  
    doc_checks_prefix="DOC",  
    entity_types=["claimant", "vendor"],  
)
```

`claim_types` drives the frontend tab list and color map. `entity_types` drives the column headers in the case queue.

Step 3 — Implement `generate_fn`

```
def generate_my_domain_data() -> dict:  
    """Return a dict whose top-level keys match context_keys exactly."""  
    claims = [...]  
    claimants = [...]  
    vendors = [...]  
    return {  
        "claims": claims,  
        "claimants": claimants,  
        "vendors": vendors,  
    }
```

Every key listed in `context_keys` must appear as a top-level key of this dict. The conformance test (`test_plugin_conformance.py`) verifies this.

Step 4 — Implement `rules_fn` using `core/engine.py`

```
from core.engine import RuleSpec, run_rules

def _r001_pred(claim, ctx) -> bool:
    return claim.claim_amount > 50_000

def _r001_explain(claim, ctx, triggered: bool) -> str:
    if triggered:
        return f"Claim amount ${claim.claim_amount:,.2f} exceeds $50,000 threshold"
    return "Claim amount within normal range"

MY_RULE_SPECS = [
    RuleSpec(
        id="R-001",
        name="High Value Claim",
        severity="FLAG",          # "BLOCK" triggers a score floor; "FLAG" adds a boost
        predicate=_r001_pred,
        explain=_r001_explain,
        details_fn=None,         # optional: (claim, ctx) -> dict of extra context
    ),
    # ... add more rules
]

def run_my_rules(claim, context_dict) -> list:
    return run_rules(claim, context_dict, MY_RULE_SPECS)
```

severity values: "BLOCK" → score floored at 70+ and increments with additional BLOCKs; "FLAG" → additive boost to risk score; "INFO" → no scoring effect.

Step 5 — Implement `score_fn` using `core/engine.py`

```
from core.engine import FeatureSpec, score_claim

def _amount_extractor(claim, ctx) -> float:
    return claim.claim_amount / 10_000.0 # raw value; normalizer_bounds will scale to [0,1]

MY_FEATURE_SPECS = [
    FeatureSpec(
        category="financial",
        name="Claim Amount",
        weight=0.5,                      # must sum to 1.0 across all specs
        extractor=_amount_extractor,
        normalizer_bounds=(0.0, 5.0),   # raw [0, 5] → normalised [0, 1]
        factor_fn=None,                 # optional: (claim, ctx, raw) -> str | None
    ),
    # ... add more features
]

def score_my_claim(claim, context_dict, rules_results) -> object:
    return score_claim(
        claim, context_dict, rules_results,
        feature_specs=MY_FEATURE_SPECS,
        risk_tiers=MY_DOMAIN_CONFIG.risk_tiers,
    )
```

Feature weights should sum to 1.0. The engine applies feature scores at 40% weight and rules boost at 60% weight by default (configurable via kwargs to `score_claim`).

Step 6 — Implement `build_queue_fn`

```
from cases import Case, CaseType, ClaimType, CasePriority, CaseStatus
from datetime import datetime

def build_my_queue(claims, risk_scores, rules, data) -> list:
    queue = []
    claimant_map = {c.claimant_id: c for c in data.get("claimants", [])}

    for claim in claims:
        score = risk_scores.get(claim.claim_id)
        if not score:
            continue
        priority = CasePriority[score.tier] # HIGH/MEDIUM/LOW -> enum
        claimant = claimant_map.get(claim.claimant_id)

        case = Case(
            case_id=claim.claim_id,
            case_type=CaseType.CAR_INSURANCE, # use the enum value that fits your domain
            claim_type=ClaimType(claim.claim_type),
            subject_id=claim.claimant_id,
            subject_name=claimant.name if claimant else "",
            asset_description=getattr(claim, "asset_description", ""),
            counterparty_name=getattr(claim, "vendor_name", ""),
            risk_score=score.total_score,
            priority=priority,
            flag_reason="; ".join(
                r.explanation for r in rules.get(claim.claim_id, []) if r.triggered
            ),
            claim_amount=claim.claim_amount,
        )
        queue.append(case)

    return sorted(queue, key=lambda c: c.risk_score, reverse=True)
```

Critical: Never add `member_id`, `employer_name`, or `provider_name` to a `Case` object. Map your domain's entity IDs to the neutral `subject_id`, `asset_description`, and `counterparty_name`.

Step 7 — Implement `checklist_module`

The module must expose three names: `run_full_checklist`, `run_checklist_step`, and `CHECKLIST_STEPS`.

```

# my_domain_checklist.py
from core.engine import Step, ChecklistStepResult, run_checklist, run_checklist_step_spec
from typing import Dict, List

def _initial_review(claim, context: Dict) -> ChecklistStepResult:
    findings = []
    ok = True
    if not getattr(claim, "claimant_id", None):
        findings.append("MISSING: Claimant ID"); ok = False
    else:
        findings.append("Claimant ID present")
    return ChecklistStepResult(1, "Initial Review", "pass" if ok else "needs_review", ok, findings)

def _document_check(claim, context: Dict) -> ChecklistStepResult:
    # ... your logic
    pass

def _final_determination(claim, context: Dict) -> ChecklistStepResult:
    # ... your logic
    pass

CHECKLIST: List[Step] = [
    Step(id="initial_review", name="Initial Review", check=_initial_review),
    Step(id="document_check", name="Document Check", check=_document_check),
    Step(id="final_determination", name="Final Determination", check=_final_determination),
]

# MUST be derived from CHECKLIST – never hardcoded separately
CHECKLIST_STEPS = [(i + 1, step.name) for i, step in enumerate(CHECKLIST)]

def run_checklist_step(step_number: int, claim_id: str, context: Dict) -> ChecklistStepResult:
    claim = next((c for c in context.get("claims", []) if c.claim_id == claim_id), None)
    if not claim:
        name = CHECKLIST[step_number - 1].name if 0 < step_number <= len(CHECKLIST) else "Unknown"
        return ChecklistStepResult(step_number, name, "error", False, ["Claim not found"])
    result = run_checklist_step_spec(CHECKLIST[step_number - 1], claim, context)
    result.step_number = step_number
    return result

def run_full_checklist(claim_id: str, context: Dict) -> List[ChecklistStepResult]:
    results = run_checklist(CHECKLIST, claim_id, context)
    for i, r in enumerate(results):
        r.step_number = i + 1
    return results

```

Step 8 — Implement agent tools

Each tool module needs a `set_context(data_ctx)` function and a list of LangChain tools:

```

# tools_my_domain.py
from langchain_core.tools import tool

_context = None

def set_context(ctx):
    global _context
    _context = ctx

@tool
def get_claim_details(claim_id: str) -> str:
    """Return details for a specific claim."""
    if not _context:
        return "Context not loaded"
    claim = next((c for c in _context.claims if c.claim_id == claim_id), None)
    return str(claim) if claim else f"Claim {claim_id} not found"

MY_DOMAIN_TOOLS = [get_claim_details]

```

Repeat the same pattern for investigation tools (`tools_my_domain_investigation.py`) and dossier tools (`tools_my_domain_dossier.py`). All three `set_context` functions will be passed to `set_tool_context_fns` in the PLUGIN.

Step 9 — Write the vision prompt

The vision prompt is a string fed verbatim to the vision LLM. It must request a JSON response with `checks` , `overall_assessment` , and `confidence` keys:

```
MY_VISION_PROMPT = """\n
You are a claims forensics analyst. Examine the submitted evidence image and
evaluate it against ALL checks below.

DOC-001: Image is clear and unaltered
DOC-002: Date/time metadata is consistent with claim date
DOC-003: Asset matches the claimed description
...

RESPOND WITH ONLY valid JSON – no markdown fences, no extra text.
{
  "checks": [
    {"id": "DOC-001", "result": "pass|fail|unclear", "note": "..."},
    ...
  ],
  "overall_assessment": "pass|fail|needs_review",
  "confidence": 0.0
}
"""
```

Step 10 — Implement `build_checklist_ctx` and `dossier_context_fn`

These two functions bridge the generic `DataContext` (from `main.py`) to the flat dicts your checklist and dossier logic expect.

```
def _build_checklist_ctx(data_ctx, claim_id: str) -> dict:
    """Return the flat dict your checklist step functions receive as `context`."""
    return {
        "claims": data_ctx.claims,
        "claimants": data_ctx.entities.get("claimants", []),
        "vendors": data_ctx.entities.get("vendors", []),
        "claim_rules": data_ctx.claim_rules,
        "claim_risk_scores": data_ctx.claim_risk_scores,
        "claim_doc_results": data_ctx.claim_doc_results,
        "has_claim_images": False,
    }

def _dossier_context_fn(data_ctx, claim_id: str) -> dict:
    """Return the dict the dossier LLM prompt builder reads."""
    claim = data_ctx.get_claim(claim_id)
    subject_id = getattr(claim, "claimant_id", "") if claim else ""
    return {
        "claim": claim,
        "subject_id": subject_id,
        "risk": data_ctx.claim_risk_scores.get(claim_id),
        "rules": data_ctx.claim_rules.get(claim_id, []),
        "doc_results": data_ctx.claim_doc_results.get(claim_id, []),
        "tasks": data_ctx.get_claim_tasks(claim_id),
        "all_claims": data_ctx.claims,
    }
```

The keys you return here must match what your copilot / investigation / dossier prompt templates reference.

Step 11 — Write the agent prompts

Three prompts are required under the `investigation_prompts` dict: `ORCHESTRATOR`, `INVESTIGATION`, and `DOSSIER`. One additional prompt for the copilot:

```
MY_COPILOT_PROMPT = """\n
You are an AI claims examiner assistant for My Domain.\n
You have access to tools to look up claims, claimants, and vendors.\n
...\n
"""\n

MY_ORCHESTRATOR_PROMPT = "...\" # decides which sub-agent to route to\n
MY_INVESTIGATION_PROMPT = "...\" # drives the investigation agent\n
MY_DOSSIER_PROMPT = "...\" # drives the dossier-building agent
```

Reference the car equivalents in `agents/prompts_car.py` and `agents/prompts_investigation_car.py` as templates.

Step 12 — Assemble `PLUGIN` in `domains/my_domain/__init__.py`

```
from core.contracts import DomainPlugin\n
from core.registry import register_plugin\n
import my_domain_checklist as _checklist_mod\n
from tools_my_domain import MY_DOMAIN_TOOLS, set_context as _set_ctx\n
from tools_my_domain_investigation import MY_INV_TOOLS, set_context as _set_inv_ctx\n
from tools_my_domain_dossier import MY_DOS_TOOLS, set_context as _set_dos_ctx\n

PLUGIN = DomainPlugin(\n
    name="my_domain",\n
    config=MY_DOMAIN_CONFIG,\n

    generate_fn=generate_my_domain_data,\n
    rules_fn=run_my_rules,\n
    score_fn=score_my_claim,\n
    build_queue_fn=build_my_queue,\n
    checklist_module=_checklist_mod,\n

    copilot_prompt=MY_COPILOT_PROMPT,\n
    copilot_tools=MY_DOMAIN_TOOLS,\n

    investigation_prompts={\n
        "ORCHESTRATOR": MY_ORCHESTRATOR_PROMPT,\n
        "INVESTIGATION": MY_INVESTIGATION_PROMPT,\n
        "DOSSIER": MY_DOSSIER_PROMPT,\n
    },\n
    investigation_tools=MY_INV_TOOLS,\n
    dossier_tools=MY_DOS_TOOLS,\n

    vision_prompt=MY_VISION_PROMPT,\n
    checklist_descriptions={\n
        1: "Initial review of claim completeness",\n
        2: "Document and evidence check",\n
        3: "Final determination for examiner",\n
    },\n

    context_keys=["claims", "claimants", "vendors"],\n
    build_checklist_ctx=_build_checklist_ctx,\n
    dossier_context_fn=_dossier_context_fn,\n
    set_tool_context_fns=[_set_ctx, _set_inv_ctx, _set_dos_ctx],\n
    entity_types=MY_DOMAIN_CONFIG.entity_types,\n
)\n

register_plugin(PLUGIN)
```

Step 13 — Register in `domains/__init__.py`

```
import domains.car          # noqa: F401
import domains.my_domain    # noqa: F401 ← add exactly this line
```

Step 14 — Set env var and restart

```
# .env
DOMAIN_MODE=my_domain
```

```
# Windows PowerShell
$env:DOMAIN_MODE="my_domain"
python -m uvicorn api:app --host 0.0.0.0 --port 8000
```

Step 15 — Verify with the conformance and swap tests

```
pytest tests/test_plugin_conformance.py
$env:DOMAIN_MODE="my_domain"; pytest tests/test_domain_swap.py
```

If both pass, the domain is correctly wired. The conformance test asserts:

- All `DomainPlugin` fields present and typed correctly
- `generate_fn` returns all declared `context_keys`
- `rules_fn` returns a valid `List[RuleResult]`
- `score_fn` returns `RiskBreakdown` with `total_score` in `[0, 100]` and a valid tier
- `Case` objects have neutral fields and no phantom fields

Important Constraints & Gotchas

Circular import bootstrap

`core/registry.py` is pure — it imports nothing from `domains/`. The bootstrap lives in `domains/__init__.py`. Any module that calls `get_active_plugin()` **at import time** will fail with `KeyError` because the registry is empty while domain modules are loading. Always defer plugin access into a function body:

```
# WRONG — registry is empty at module load time:
_VISION_PROMPT = get_active_plugin().vision_prompt

# CORRECT — deferred to first actual call:
def _get_vision_prompt():
    return get_active_plugin().vision_prompt
```

This pattern is used in `agents/nodes.py` (lazy helpers `_get_orchestrator_prompt()`, etc.) and `intelligence/document_vision.py` (`_get_vision_prompt()` with a `_CAR_VISION_PROMPT` fallback during

bootstrap).

Case phantom fields

Never add `member_id`, `employer_name`, or `provider_name` to a `Case` object. These are legacy field names from a prior health-insurance domain. The neutral equivalents are `subject_id` / `subject_name`, `asset_description`, and `counterparty_name`. The conformance test (`test_queue_cases_no_phantom_fields`) will catch violations.

Cache stamping

The pickle payload is a 2-tuple `(domain_name, ctx)`. Loading a bare `DataContext` (old format) or a tuple with a different `domain_name` triggers automatic regeneration. Delete `data_cache_*.pkl` files manually to force a full rebuild.

`ClaimType` enum — extend for new domains

`cases.py` defines `ClaimType` with the car insurance values. When adding a domain with different claim types, add its values to the enum:

```
class ClaimType(str, Enum):
    # Car Insurance
    COLLISION = "collision"
    # ...
    # My Domain
    TYPE_A = "type_a"
    TYPE_B = "type_b"
```

Frontend is fully config-driven

All claim type labels, colors, tab lists, and entity column headers come from `/api/config`. Do **not** add domain-specific arrays to React components. `/api/config` derives these from `DomainConfig.claim_types`, `DomainConfig.entity_types`, and the `type_labels` / `type_colors` maps in `api.py`.

Model ID

The canonical model is `us.anthropic.claude-haiku-4-5-20251001-v1:0`. Set `BEDROCK_MODEL_ID` in `.env`. Use `BEDROCK_VISION_MODEL_ID` to override for vision checks only.

Windows console encoding

If running on Windows, set `PYTHONIOENCODING=utf-8` before starting the server to prevent `UnicodeEncodeError` on Unicode characters in console output:

```
$env:PYTHONIOENCODING = "utf-8"
python -m uvicorn api:app --host 0.0.0.0 --port 8000
```

Engine API Reference (`core/engine.py`)

`run_rules(claim, context, specs) -> List[RuleResult]`

Param	Type	Description
<code>claim</code>	any	Domain claim object
<code>context</code>	<code>Dict</code>	The context dict built from <code>context_keys</code>
<code>specs</code>	<code>List[RuleSpec]</code>	Your domain's rule specifications

Returns one `RuleResult` per spec. Catches exceptions per-rule and returns an `"ERR"` result instead of crashing.

`score_claim(claim, context, rules_results, *, feature_specs, risk_tiers, ...) -> RiskBreakdown`

Key kwargs (all have defaults matching the car domain calibration):

Kwarg	Default	Meaning
<code>feature_weight</code>	<code>0.40</code>	Feature side contribution to final score
<code>rules_weight</code>	<code>0.60</code>	Maximum rules-boost contribution
<code>block_boost</code>	<code>0.20</code>	Per-BLOCK rule boost
<code>flag_boost</code>	<code>0.08</code>	Per-FLAG rule boost
<code>block_floor_base</code>	<code>70.0</code>	Minimum score when any BLOCK fires
<code>photo_floor</code>	<code>90.0</code>	Minimum score when photo evidence present

`run_checklist(steps, claim_id, context) -> List[ChecklistStepResult]`

Iterates `steps` in order, calls each `Step.check(claim, context)`, and catches per-step exceptions. Returns one `ChecklistStepResult` per step.

Result types

```
RuleResult:      rule_id, rule_name, severity, triggered, explanation, details
RiskBreakdown:  total_score, tier, feature_contributions, top_factors, rules_boost
FeatureContribution: category, feature_name, raw_value, normalized, weight, contribution
ChecklistStepResult: step_number, step_name, status, auto_passed, findings, details
```

Test Suite Summary

File	Count	What it checks
<code>test_plugin_conformance.py</code>	24 tests	Every registered plugin satisfies the full contract
<code>test_domain_swap.py</code>	10 tests	initialize_data + queue + no phantom fields for active domain
<code>test_golden_baseline.py</code>	14 tests	Hero-case scores/tiers/rules must not drift (COL-107, THEFT-009, LIAB-021)
<code>test_rules_engine.py</code>	25 tests	Car rules R-001 to R-006 unit tests with importlib bypass for boto3

When you add a new domain, `test_plugin_conformance.py` automatically picks it up — no changes to the test file required.